Crashlytics Full Triage — 2026-06-24

Pull method: Firebase MCP (`crashlytics_get_report` + `crashlytics_batch_get_events`)
Window: 2026-05-25 → 2026-06-24 (30 days)
Apps queried: 4 (client release, client debug, api-app release, api-app debug)
Pull timestamp: 2026-06-24T (today's session)
Current build at pull time: client 1.3.11-1073 / api-app 0.2.11-19 (shipping; in-cycle)

Per §11.4.6: proven root causes are stated as facts based on stack evidence. Anything not directly confirmed by the event data is marked `UNCONFIRMED:`.

Apps With Zero Issues in the 30-Day Window

App Variant	App ID (suffix)	Issues
Client DEBUG	…54ca2ca3…	**0 — no data**
API-app RELEASE	…d57b960e…	**0 — no data**
API-app DEBUG	…2932451e…	**0 — no data**

Interpretation: debug variants have negligible real-device install base (only local testing). API-app Crashlytics is misattributed to the client release app ID — the "Defect B" telemetry misattribution documented in `.lava-ci-evidence/crashlytics-resolved/2026-06-14-defect-b-telemetry-attribution-findings.md`. The two api-app crash issues (9ba8502e, b9baeaede) appear under the **client release** app ID because api-app's `google-services.json` currently maps to the client Firebase app.

Client RELEASE (`digital.vasic.lava.client`) — 8 Issues in Window

Total issues returned: 8
Device in every event: Samsung SM-S918B (Galaxy S23 Ultra), Android 16, ARM64
This is 1 real-world tester device (installationUuid `B5579AE9…` appears in most events).

Issue Table

#	Issue ID (short)	Title / Top Frame	Type	Events	Users	First Seen
1	`58a1335272bc`	CredentialsKeyHolder.require — locked	FATAL	5	1	1.3.10
2	`47b000d54ff6`	ProviderCatalogRepository — HTTP 401	NON_FATAL	6	1	1.3.4
3	`9ba8502ee0ba`	ApiEngineService.onStartCommand — FGS start not allowed	FATAL	5	1	0.2.6
4	`c7c8cccad09f`	LazyColumn nested in verticalScroll — infinite constraint	FATAL	2	1	1.2.3
5	`042b9b611cf1`	TLS CertPathValidatorException — self-signed cert	NON_FATAL	1	1	1.3.3
6	`3937b7f08628`	SSE UnknownHost — lava-api.local unresolvable	NON_FATAL	1	1	1.3.0
7	`8cde0ac208b3`	TopicPageDto MissingFieldException — API schema mismatch	NON_FATAL	1	1	1.3.9
8	`b9baeaede585`	ApiEngineService FGS did not stop in time	FATAL	1	1	0.2.6

Per-Issue Deep-Dive

Issue 1 — `58a1335272bc` — CredentialsKeyHolder FATAL (🔴 HIGHEST PRIORITY)

Classification: NEW/OPEN — critical FATAL crash, 5 events, 1 user, current version 1.3.10

Stack (from sample event `6A3443DF…`, 2026-06-18T19:16:16Z, v1.3.10-1067 release):

```
java.lang.IllegalStateException: credentials key holder is locked
— prompt user for passphrase first
  at lava.credentials.session.CredentialsKeyHolder.require
(CredentialsKeyHolder.java:23)
  at
lava.credentials.di.CredentialsModule.provideCredentialsKeyProvide
(CredentialsModule.java:77)
  at lava.credentials.CredentialsEntryRepositoryImpl.decode
(CredentialsEntryRepositoryImpl.kt:78)
  at lava.credentials.CredentialsEntryRepositoryImpl$observe$
$inlined$map$1$2.emit (CredentialsEntryRepositoryImpl.java:51)
  at androidx.room.coroutines.FlowUtil$createFlow$
$inlined$map$1$2.emit (FlowUtil.java:223)
```

```
  at [Room Flow continuation chain]
  at android.os.Looper.loop (Looper.java:392)
  at android.app.ActivityThread.main
(ActivityThread.java:10346)    ← MAIN THREAD CRASH
```

Root cause (from stack): `CredentialsKeyHolder.require()` at line 23 throws unconditionally when the key holder is in the locked state. The call chain is: Room DB Flow emission → `CredentialsEntryRepositoryImpl.decode()` → `CredentialsModule.provideCredentialsKeyProvider$lambda$2()` → `CredentialsKeyHolder.require()`. This is invoked on the **main thread** via a Room-emitting coroutine dispatched onto the main Looper. The key holder becomes locked when the user's session is in a state where the passphrase has not yet been provided (initial launch, session expiry, or app restore after process death). Crucially, this is not an edge-case: the breadcrumb shows two search submits (`lava_search_submit: mumy`, then `lava_search_submit: prince`) immediately preceding the crash — the user is actively using search, the credentials flow is observed in the background, and the observation of the (still-locked) key holder triggers the throw.

Impact: FATAL on main thread → full app crash. Occurred in 1.3.10 (current). 5 events across the session indicate repeated restarts then re-crash.

Action required:

- `CredentialsKeyHolder.require()` must not throw on the main thread when called from a Room Flow observer. The `decode()` function in `CredentialsEntryRepositoryImpl.kt:78` must guard against the locked state and return a sentinel (null / empty list) rather than propagating the throw into the main-thread Looper.
- A `§6.0` closure log must be authored when the fix lands.
- A regression Challenge Test is required: deliberately lock the key holder, trigger a search, assert the app does NOT crash and shows an appropriate re-auth prompt.

Issue 2 — `47b000d54ff6` — ProviderCatalog HTTP 401 NON_FATAL

Classification: ENVIRONMENTAL / monitored — behavior is by-design, telemetry severity is appropriate

Stack (from sample event `6A2D984F…`, 2026-06-13T17:51:55Z, v1.3.5-1062 release):

```
java.lang.IllegalStateException: provider discovery failed: HTTP
401 for https://192.168.0.107:8443/providers
  at
lava.data.provider.ProviderCatalogRepository$fetchProviders$2.invol
(ProviderCatalogRepository.kt:112)

customKeys.error = provider_catalog_fetch_failed
```

Root cause: The client app calls `GET /providers` against the discovered LAN API (`192.168.0.107:8443`) with no auth token (or an expired/missing API key). The lava-api-go endpoint at `/providers` requires authentication from 1.3.x onward. The `ProviderCatalogRepository` correctly records this as a non-fatal and falls back to bundled providers.

Prior analysis: Documented in `.lava-ci-evidence/crashlytics-resolved/2026-06-13-providers-catalogue-401-auth-gated.md`. That log concluded the 401 is expected when the api-key exchange hasn't completed at first onboarding (race between API selection and the key-handshake). The last-seen version is 1.3.5 (before the auth-key flow was hardened in 1.3.7+). This issue has not recurred in 1.3.7–1.3.10.

Action: Monitor; no code fix owed. If it reappears on 1.3.11+, the api-key provisioning flow needs investigation. The console state is OPEN because it was never close-marked.

Issue 3 — `9ba8502ee0ba` — ApiEngineService FGS Start Not Allowed FATAL

Classification: ALREADY-FIXED — fix commit `ed03cac2` (specialUse FGS type)

Stack (from sample event `6A2E6C61…`, 2026-06-14T08:54:57Z, api-app 0.2.6-10):

```
android.app.ForegroundServiceStartNotAllowedException:
  Service.startForeground() not allowed due to
mAllowStartForeground false:
  digital.vasic.lava.api/lava.api.app.service.ApiEngineService
  at lava.api.app.service.ApiEngineService.onStartCommand
(ApiEngineService.kt:113)
```

`customKeys.artifact = api-app`, `version_code = 10` (0.2.6)

Root cause: Android 14+ `dataSync` FGS type has a 6h cumulative runtime budget. After

> 6h uptime the OS disallows new `startForeground()` calls. First-seen and last-seen are both `0.2.6` (api-app code 10). Fixed in api-app `0.2.8-12` by migrating to `specialUse` FGS type.

Prior closure log: `.lava-ci-evidence/crashlytics-resolved/2026-06-14-apiapp-fgs-datasync-budget.md`

Note: This event appears under the client release app ID due to the Defect B misattribution (`google-services.json` in api-app pointing to the client Firebase project).

Action: None — fix shipped in api-app 0.2.8. Operator should close-mark in Firebase Console once 0.2.11-19 is distributed and observed clean.

Issue 4 — `c7c8cccad09f` — LazyColumn in verticalScroll FATAL (🟠 OPEN — REGRESSION)

Classification: NEW/OPEN — expected to be fixed but still seen on 1.3.10

Stack (from sample event `6A2EFA57…`, 2026-06-14T19:01:27Z, v1.3.10-1067 release):

```
java.lang.IllegalStateException: Vertically scrollable component
was measured with an
infinity maximum height constraints, which is disallowed. One of
the common reasons is
nesting layouts like LazyColumn and
Column(Modifier.verticalScroll()).
  at
androidx.compose.foundation.internal.InlineClassHelperKt.throwIlleg
  at
androidx.compose.foundation.CheckScrollableContainerConstraintsKt.c
  at
androidx.compose.foundation.lazy.LazyListKt$rememberLazyListMeasure
```

Root cause: A `LazyColumn` is rendered inside a parent with `Modifier.verticalScroll()` (or equivalent unbounded-height parent). This is the §6.Q forbidden antipattern. This issue was first documented in the `.lava-ci-evidence/crashlytics-resolved/2026-05-05-tracker-settings-nested-scroll.md` closure log (2026-05-05, the TrackerSelectorList fix). However, the current sample event is from **1.3.10-1067** (2026-06-14), which is after that fix. Two possibilities:

1. The fix for `TrackerSelectorList` landed but a DIFFERENT screen also has the same nesting antipattern and was not addressed (e.g. another settings screen, the credentials manager screen, or a newly added screen after SP-3a).
2. The original TrackerSelectorList fix was incomplete (the fix patched one code path but not all invocations of that composable).

The `firstSeenVersion = 1.2.3` and `lastSeenVersion = 1.3.10` with the issue still OPEN confirms this recurs across many versions and has NOT been fully resolved. The breadcrumb shows `screen_view{MainActivity}` only — no navigation breadcrumb to which specific sub-screen triggered it.

Impact: FATAL crash. The user triggered it from an unidentified screen in 1.3.10. Needs a full codebase scan for remaining `LazyColumn`/`LazyVerticalGrid` inside `verticalScroll` parents (the §6.Q structural test was written for `TrackerSelectorList` specifically but may not cover all newly added composables).

Action required:

- Run the §6.Q structural test suite against all composables; find the CURRENT offending screen.
- Fix the remaining nested-scroll violation(s).
- Add a specific regression Challenge Test for the screen that produced this 1.3.10 event.
- Author a §6.O closure log when the fix is confirmed clean.

Issue 5 — `042b9b611cf1` — TLS CertPathValidatorException NON_FATAL

Classification: ENVIRONMENTAL — self-signed LAN certificate, expected on first-connection before cert pinning setup

Stack (from sample event `6A2AC47F…`, 2026-06-11T14:22:01Z, v1.3.3-1060 release):

```
javax.net.ssl.SSLHandshakeException:
java.security.cert.CertPathValidatorException:
  Trust anchor for certification path not found.
  at okhttp3.internal.connection.RealConnection.connectTls
(RealConnection.kt:379)
```

`customKeys.error = provider_catalog_fetch_failed` — this is the provider-catalog discovery call.

Root cause: The lava-api-go server uses a self-signed TLS certificate for the LAN HTTPS endpoint. The Android TrustManager (system trust store) does not accept self-signed certs by default. The `ProviderCatalogRepository` attempts HTTPS to `192.168.x.x:8443` before the user has added a cert trust exception or before the app's `network_security_config.xml` adds the server's certificate. This is documented in `.lava-ci-evidence/crashlytics-resolved/2026-06-12-provider-catalog-fetch-tls.md`.

Last seen: 1.3.3 (not seen since). The TLS trust was presumably resolved in the user's session after initial setup.

Action: Monitor; no code fix owed for this specific occurrence. If recurring in 1.3.11+, the onboarding cert-trust flow needs investigation. Console state is OPEN — operator should close-mark.

Issue 6 — `3937b7f08628` — SSE UnknownHost `lava-api.local` NON_FATAL

Classification: ENVIRONMENTAL / noise — expected when API server is not on LAN

Stack (from sample event `6A20DF91…`, 2026-06-04T02:16:51Z, v1.3.0-1057 release):

```
java.lang.IllegalStateException: SSE error: Connection failed:
  Unable to resolve host "lava-api.local": No address associated
with hostname
  at
lava.search.result.SearchResultViewModel$observeSseSearch$1$2.emit
```

```
customKeys.query = prince
```

Root cause: mDNS `lava-api.local` cannot be resolved when the user's device is off the LAN where the lava-api-go server is running (e.g. mobile data, different WiFi, or server not started). The `SearchResultViewModel` treats this as a non-fatal.

Documented in `.lava-ci-evidence/crashlytics-resolved/2026-06-13-sse-host-resolve-telemetry-severity.md`. The conclusion from that log: telemetry severity is appropriate (non-fatal, not crash), but the UX should show a clear "API not reachable" message rather than a generic error.

Last seen: 1.3.0 (one event only). Not recurring in 1.3.1+.

Action: No code fix required for this specific Crashlytics issue. The UX improvement (better error message for offline state) is a product enhancement, not a crash fix.

Issue 7 — `8cde0ac208b3` — TopicPageDto MissingFieldException NON_FATAL (🟠 NEW — Schema Contract Break)

Classification: NEW/OPEN — API response schema mismatch, topic loading broken for certain content

Stack (from sample event `6A2ED1B3…`, 2026-06-14T16:09:23Z, v1.3.9-1066 release, Genymobile Pixel 9):

```
io.ktor.serialization.JsonConvertException: Illegal input:
  Fields [id, title, author, category, torrentData, commentsPage]
are required for
  type with serial name 'lava.network.dto.topic.TopicPageDto', but
they were missing at path: $
  at
io.ktor.serialization.kotlinx.KotlinxSerializationConverter.deseria
  Caused by: kotlinx.serialization.MissingFieldException
  at lava.network.dto.topic.TopicPageDto.<init>
(TopicPageDto.kt:6)

customKeys.error = load_topic_failed
customKeys.topic_id = WPO-20230122202907-crawl897
```

Breadcrumb log: `lava_view_topic { topic_id: WPO-20230122202907-crawl897 }` — the user tapped a specific topic ID starting with prefix `WPO-`.

Root cause (from stack): The topic response JSON returned for `WPO-20230122202907-crawl897` does not contain the expected top-level fields (`id`, `title`, `author`, `category`, `torrentData`, `commentsPage`). The `TopicPageDto` is a strict deserialization target with no `@Serializable` defaults for these required fields. When the JSON response has a different root shape (e.g. an error envelope, a redirect body, or a non-standard topic format from the `crawl` provider), the deserialization throws `MissingFieldException`.

The topic ID prefix `WPO-20230122202907-crawl897` indicates this is an **Internet Archive / crawl provider** topic (not RuTracker or RuTor), based on the date-stamped crawl ID format. The `lava-api-go` proxy's response for this topic type likely returns a different JSON structure than `TopicPageDto` expects — either because the crawl provider's schema has different field names, or because the response is an error/empty object wrapped differently.

Impact: NON_FATAL, but the user sees a "topic load failed" error for any Internet Archive crawl topic they try to open. This renders the Internet Archive provider's content unviewable for topics with the `crawl` ID format.

Action required:

- Inspect `lava-api-go`'s `/topic/{id}` response for `WPO-*` (Internet Archive crawl) IDs — compare the actual JSON shape to `TopicPageDto`'s required fields.
- Either: (a) make the missing fields optional with `@SerialName` + defaults in `TopicPageDto`; or (b) add a provider-specific DTO for Internet Archive crawl topics; or (c) fix the lava-api-go response to always include the required fields.
- A regression Challenge Test is required: load a `WPO-*` topic, assert the topic page renders (or shows a graceful error), not a crash/non-fatal.
- Author a §6.O closure log.

Issue 8 — `b9baeaede585` — ApiEngineService FGS Did Not Stop In Time FATAL

Classification: ALREADY-FIXED — same root cause as Issue 3, companion crash; fix commit `ed03cac2`

Stack (from sample event `6A2DB3C7…`, 2026-06-14T01:47:53Z, api-app 0.2.6-10):

```
android.app.RemoteServiceException$ForegroundServiceDidNotStopInTir
  A foreground service of type dataSync did not stop within its
timeout:
  ComponentInfo{digital.vasic.lava.api/
lava.api.app.service.ApiEngineService}
  at
android.app.ActivityThread.generateForegroundServiceDidNotStopInTir
```

Root cause: Companion crash to Issue 3 — when the 6h `dataSync` budget is exhausted and the service fails to start, Android also kills it with `ForegroundServiceDidNotStopInTimeException` if the service doesn't exit quickly. Both issues share the same fix (`specialUse` FGS type in `ed03cac2`).

Note: Misattributed to client Firebase app ID (Defect B, as with Issue 3).

Action: None — fix shipped in 0.2.8. Same close-mark action as Issue 3.

Summary by Classification

Classification	Count	Issue IDs
NEW/OPEN — needs fix	**3**	`58a13352` (P0 FATAL), `c7c8ccca` (P1 FATAL recurring), `8cde0ac2` (P2 NON_FATAL)
ALREADY-FIXED	2	`9ba8502e`, `b9baeae5` (both fixed in api-app 0.2.8 / `ed03cac2`)

Classification	Count	Issue IDs
ENVIRONMENTAL / monitored	2	`47b000d5` (401 auth-gated catalog), `042b9b61` (TLS self-signed)
ENVIRONMENTAL / noise	1	`3937b7f0` (mDNS resolve on off-LAN)

Priority Action List

P0 — FIX NOW: Issue 1 — CredentialsKeyHolder locked crash (`58a13352`)

- **Why P0:** 5 FALTALs in 1.3.10 (current release). Main thread crash. User was actively using search when it hit.
- **Fix:** Guard `CredentialsEntryRepositoryImpl.decode()` against the locked key holder state — return empty/null sentinel instead of propagating the throw. Add a passphrase-prompt side effect.
- **Files:** `submodules/security/src/.../CredentialsKeyHolder.java:23`, `CredentialsEntryRepositoryImpl.kt:78`, `CredentialsModule.java:77`
- **Test owed:** Challenge test that locks key holder, submits search, verifies app stays alive and re-auth prompt appears.

P1 — FIX THIS CYCLE: Issue 4 — LazyColumn in verticalScroll recurring (`c7c8ccca`)

- **Why P1:** FATAL crash recurring through 1.3.10 despite a prior 1.2.3-era fix. There is an unresolved second nesting site still in production code.
- **Fix:** Run §6.Q scanner across ALL composables written/added since 1.2.5; find the current offending screen; apply the bounded-height or item()-header pattern.
- **Test owed:** §6.Q structural test must cover the offending screen. Challenge test must drive navigation to that screen and verify it renders.

P2 — FIX NEXT CYCLE: Issue 7 — TopicPageDto schema mismatch (`8cde0ac2`)

- **Why P2:** NON_FATAL, but Internet Archive crawl topics are silently unviewable. Impact is limited to one content provider but affects a user who browsed to a `WPO-*` topic.
- **Fix:** Make `TopicPageDto` fields optional OR fix the lava-api-go response shape for crawl topics.
- **Test owed:** Challenge test loading a crawl-format topic.

Highest-Impact Unaddressed Issue

Issue 1 — `58a1335272bc4ee06595bda6302a670a` — CredentialsKeyHolder locked FATAL

5 crashes in the **current release version (1.3.10)** from a single active user. Main-thread crash triggered during normal search use. No prior cycle has addressed this. This is the single bug most likely to cause the operator's tester to file "app crashes on me" in the next session.

Notes on Scope

- The known SocketTimeout issue (`9d4ad2f4d1a8b8697b1506402e045b81`) is **not in the top-8 for the 2026-05-25 → 2026-06-24 window** — it did not appear in this 30-day pull's results. The fix commits (`20d98914` + `0e81730b` + `1aa42536` + `3c1fa159`) are shipping in 1.3.11-1073. The absence of this issue from the window confirms the fix cycle is contemporary with the window.
- All 8 issues involve **1 real-device user** (Galaxy S23 Ultra / Android 16), consistent with a single tester. The operator's own device.
- The 3 other Firebase app variants (client debug, api-app release, api-app debug) returned no data in the 30-day window.